

50.050/50.550 – Advanced Algorithms

January–April Term, 2026

Homework Set 2

Due by: Week 3 Friday (13 February 2026) 1pm.

Please submit your homework online via eDimension.

Note: This homework set focuses on proofs. Remember, you would want your proof to be absolutely correct without any doubt, and not just “sound correct”, so think about how to make your proof precise. We have discussed several proofs in Week 2; try to emulate the level of details given in these proofs.

Question 1. Let $a_0, a_1, a_2, \dots$ be an infinite sequence of integers indexed by $\mathbb{N}$, such that $a_0 = 5$, $a_1 = 4$, and $a_n = 3a_{n-1} + 10a_{n-2}$ for all $n \geq 2$. Prove using induction or a variant of induction that a_n can be computed via the formula

$$a_n = 2(5^n) + 3(-2)^n$$

for all $n \in \mathbb{N}$.

[5 marks]

Note: This question is meant to help you practise writing induction arguments. No credit will be given if you do not use induction or a variant of induction. Please state the base case(s) and the induction step very clearly. Please show as much details as possible, and please do not skip steps, otherwise you may not get full credit.

Solution. Let $P(n)$ be the statement that $a_n = 2(5^n) + 3(-2)^n$ for a given natural number n .

Base cases: $P(0)$ and $P(1)$ are true, since $a_0 = 2 \cdot 5^0 + 3 \cdot (-2)^0 = 5$, and $a_1 = 2 \cdot 5^1 + 3 \cdot (-2)^1 = 4$.
[2 marks]

Induction step: Assume $P(n)$ and $P(n+1)$ are true for any natural number n . Then by induction hypothesis, we have

$$\begin{aligned} a_n &= 2(5^n) + 3(-2)^n \\ a_{n+1} &= 2(5^{n+1}) + 3(-2)^{n+1}, \end{aligned}$$

hence by the definition of a_{n+2} , we get

$$\begin{aligned} a_{n+2} &= 3a_{n+1} + 10a_n \\ &= 6(5^{n+1}) + 9(-2)^{n+1} + 20(5^n) + 30(-2)^n \\ &= 6(5^{n+1}) + 9(-2)^{n+1} + 4(5^{n+1}) - 15(-2)^{n+1} \\ &= 10(5^{n+1}) - 6(-2)^{n+1} \\ &= 2(5^{n+2}) + 3(-2)^{n+2} \end{aligned}$$

Thus we have shown that $P(n) \wedge P(n+1) \rightarrow P(n+2)$ for all $n \in \mathbb{N}$.

[2 marks]

By a variant of strong induction, the statement $P(n)$ must be true for all $n \in \mathbb{N}$, so we conclude that $a_n = 2(5^n) + 3(-2)^n$ for all $n \in \mathbb{N}$.

[1 mark]

Question 2. Prove that for all non-negative integers $m \leq n$, we have the following identity:

$$\sum_{r=m}^n \binom{n}{r} \binom{r}{m} = 2^{n-m} \binom{n}{m}. \quad [5 \text{ marks}]$$

Hint: It is up to you to choose whichever proof method you want, but there is an intuitive proof using double counting.

Solution. Consider the problem of partitioning n objects into 3 categories A, B, and C, such that there are exactly m objects in category C. We could do this in two ways. [1 mark]

First, we can select a subset of our objects with any size r (between n and m inclusive) to be in categories B and C, then from this subset select m objects to be in category C. The number of ways to do this is $\sum_{r=m}^n \binom{n}{r} \binom{r}{m}$. [2 marks]

Secondly, we can also select m objects to belong in category C, then assign the remaining $n-m$ objects to categories A and B. The number of ways to do this is $\binom{n}{m} \cdot 2^{n-m}$. [2 marks]

Thus, by double counting, we have shown that $\sum_{r=m}^n \binom{n}{r} \binom{r}{m} = \binom{n}{m} \cdot 2^{n-m}$.

Question 3. There are 25 letter A's and 25 letter B's written on a whiteboard. You may erase any two letters, then write A if they are unequal and write B if they are equal. Do this until there is only one letter remaining on the whiteboard. Is it A or B? Could it be either A or B, depending on how you erased the letter? Or is this last remaining letter always the same? For each possible remaining letter, give an explicit example of how you could get that letter. Justify your answers with a proof, and show as much details as possible. [5 marks]

Solution. We first show that the parity of the number of A's on the whiteboard is invariant. [1 mark]

When we erase two equal letters and write B on the whiteboard, we either remove 0 or 2 A's, keeping the parity of the number of A's the same. When we erase two different letters and write A, exactly one of the removed letters must be A, so the number of A's remains constant. [2 marks]

Thus, regardless of what we do, the parity of the number of A's does not change, and so it is impossible to go from the starting state of 25 A's to an ending state with no A's. [1 mark]

Thus, if there only one letter remaining on the whiteboard, then this final letter must be a A. To see explicitly how we can obtain this final letter A, we could first erase 12 pairs of B's from among the 25 B's initially written on the whiteboard, which gives 12 new B's, together with 1 remaining B, for a total of 13 B's. Among these 13 B's, erase 6 pairs of B's, to get 6 new B's, together with 1 remaining B, for a total of 7 B's. Iterating this process, we would get a single remaining B. Next, we erase 12 pairs of A's from among the 25 A's initially written on the whiteboard, which gives 12 new B's, together with 1 single left-over A, and 1 single remaining B from the previous process, for a total of 13 B's and 1 A. Among the 13 B's, erase 6 pairs of B's, to get a total of 7 B's. Iterate this process, until eventually, we have just a single B and a single A on the whiteboard. Finally, erase both letters and write A as the final letter. [1 mark]

Question 4. In a magical kingdom, there are 10 castles, and 500 immortal warriors. On Day 1, each of the 500 warriors randomly chooses to defend one of the castles. Every warrior has the

option of “relocation”, which is to move from defending one castle to defending another castle, subject to the rule that relocation from castle A to castle B is allowed only if after relocating, the number of warriors at castle A would still be at least the number of warriors at castle B. From Day 2 onwards, as long as an allowable relocation is possible for at least one of the 500 warriors, there will always be some warrior who would relocate that day (while following the rule for relocation). Prove that no matter what the initial distribution of warriors was on Day 1, and no matter which castles were selected for relocation (without breaking the rule for relocation), eventually every castle would have exactly 50 warriors. [5 marks]

Solution. Consider the sum of the squares of the number of warriors at each castle. We wish to show that any relocation of warriors will only decrease this “score”. [1 mark]

Consider an arbitrary day, where a warrior relocates from castle A with a warriors to castle B with b warriors. After relocating, castle A would have $a - 1$ warriors, while castle B would have $b + 1$ warriors. By the rule for relocation, we must have $a - 1 \geq b + 1$, or equivalently, $b \leq a - 2$. Thus, the overall change in our “score” after this relocation is

$$(a - 1)^2 + (b + 1)^2 - a^2 - b^2 = 2b - 2a + 2 \leq 2(a - 2) - 2a + 2 = -2 < 0,$$

so our “score” must be strictly decreasing. [1 mark]

We also observe that unless every castle has exactly 50 warriors, there must exist a castle with either > 50 warriors or < 50 warriors. Since the average number of warriors across the castles is $\frac{500}{10} = 50$, the existence of a castle with > 50 warriors (above the average number) would force the existence of another castle with < 50 warriors (below the average number), and likewise, the existence of a castle with < 50 warriors (below the average number) would force the existence of another castle with > 50 warriors (above the average number). This means that unless every castle has exactly 50 warriors, an allowable relocation is possible for at least one of the 500 warriors. [1 mark]

Now we shall prove by contradiction that eventually every castle must have exactly 50 warriors. Assume on the contrary that the warriors never reach the configuration with every castle having exactly 50 warriors. Then, with each day, we must reach an entirely new configuration, since the “scores” of our configuration across the days are strictly decreasing. However, the total number of possible configurations (to distribute 500 warriors among 10 castles) is finite, hence we have a contradiction. [2 marks]

Question 5. (For post-graduate students only¹) [Loop invariant proofs] To ensure that an algorithm works as intended, you would need to prove the correctness of the algorithm. One common approach for proving correctness is to use a loop invariant proof.

The underlying idea is simple: Suppose your algorithm has a main for-loop or while-loop. If there is a condition P that is true before and after the execution of every iteration of the for-loop/while-loop, then this condition P is called a **loop invariant**. We can use the idea of loop invariants to prove an algorithm is correct, by identifying a suitable condition P , and showing the following:

- **Initialization:** Condition P is true prior to the first iteration of the loop.
- **Maintenance:** If P is true immediately before an iteration of the loop, then it remains true immediately after that iteration.
- **Termination:** When the loop terminates, the conditions for termination, together with P , would imply the desired outcome of the algorithm.

¹Undergraduate students may try this question, but no marks will be awarded, i.e. this will not count towards your final grade. Only feedback will be given (if any).

Formally, P is a predicate that depends on the loop variable(s). A loop variable is a variable whose value is updated within the loop. Since every for-loop can be re-expressed as a while-loop, we shall focus on loop invariants for while-loops. For example, consider the following simple algorithm to compute the maximum value of the entries of an integer array $A[1..n]$. (Following notation from the CLRS textbook, we shall write “ $A[1..n]$ ” to mean that A is an array whose first entry is $A[1]$ and whose last entry is $A[n]$; in other words, array A has n entries and the indexing for this array starts from 1.)

function MAXVALUE(A)

Require: $A[1..n]$ is an integer array

```

1:  $n \leftarrow A.length$ 
2:  $val \leftarrow -\infty$ 
3:  $i \leftarrow 0$ 
4: while  $i < n$  do
5:    $i \leftarrow i + 1$ 
6:   if  $A[i] > val$  then
7:      $val \leftarrow A[i]$ 
8: return  $val$ 
```

Given an integer array $A[1..n]$, we can compute MAXVALUE(A) to get the maximum value of the entries of A . To see why this is true, let $P = P(i, val)$ be the predicate given by

val equals the maximum value of the entries of the subarray $A[1..i]$,

where the domain of discourse for P is all pairs (i, val) satisfying $i \in \{0, 1, \dots, n\}$ and $val \in \mathbb{Z}$. (Note that i and val are variables, whose values change with each iteration of the while-loop.) For each $0 \leq j \leq n$, let m_j denote the maximum value of the entries of the subarray $A[1..j]$. Then the predicate $P(i, val)$ is identically the statement “ $val = m_i$ ”, whose truth value (i.e. true or false) will depend on the values of val and i .

- **Initialization:** Before the first iteration of the while-loop, the value of i is initialized as $i = 0$, and the value of val is initialized as $val = -\infty$. In particular, $i = 0$ means the subarray $A[1..i]$ is the empty array, so the maximum value of the entries of this empty array is $-\infty$, which is exactly the value of val (prior to the first iteration). Since the proposition $P(0, -\infty)$ is true, we infer that $P(i, val)$ is true prior to the first iteration.
- **Maintenance:** Assume that $P(i, val)$ is true immediately before the execution of an iteration of the while-loop, based on the values of i and val at the start of the iteration. Our goal is to show that $P(i, val)$ is true immediately after the execution of this iteration, based on the values of i and val at the end of the iteration.
 - $P(i, val)$ says “ $val = m_i$ ”.

Let $i = k$ and $val = v$ (for some (k, v) in the domain of discourse $\{0, 1, \dots, n\} \times \mathbb{Z}$) be the values of i and val at the start of the iteration. By assumption, $P(i, val)$ is true at the start of the iteration, which means that proposition $P(k, v)$ is true. Equivalently, we have $v = m_k$, and the proposition $P(k, v)$ is identically the proposition $P(k, m_k)$. By line 5 of the pseudocode, we know that the value of i would become $i = k + 1$ at the end of the iteration. Hence, our goal is to show that the value of val changes from $val = m_k$ at the start of the iteration, to $val = m_{k+1}$ at the end of the iteration.

Note that for any $1 \leq i \leq n$, the maximum value of the subarray $A[1..i]$ is either the maximum value m_{i-1} of the entries of the subarray $A[1..(i-1)]$, or the value of the entry

$A[i]$, whichever is larger. In other words, $m_i = \max\{m_{i-1}, A[i]\}$. Within the iteration, running lines 6 and 7 of the pseudocode would update val as follows:

$$val \leftarrow \max\{val, A[i]\}.$$

Since $val = m_k$ at the start of the iteration, we then infer that after updating val , we get $val = m_{k+1}$ at the end of the iteration. Thus, we conclude that $P(i, val) = P(k+1, m_{k+1})$ is true immediately after the execution of this iteration.

- **Termination:** We exit the while-loop after the value of i reaches $i = n$ at the end of some iteration. Let v_{final} be the value of val at the end of the last iteration. Since $P(i, val)$ is true after exiting the while-loop, it means that proposition $P(n, v_{\text{final}})$ is true. Now, proposition $P(n, v_{\text{final}})$ says that $v_{\text{final}} = m_n$, i.e. v_{final} is the maximum value of the entries of the subarray $A[1..n]$. But the subarray $A[1..n]$ is exactly the entire array A . Therefore, the value $val = v_{\text{final}}$ returned in line 6 of the pseudocode is indeed the maximum value of the entries of the input array A .

Additional resources for understanding loop invariants:

- CLRS textbook, Chapter 2.1.
- Youtube video on loop invariant proofs: https://www.youtube.com/watch?v=_maJ4Qy7Q0E
- Survey article on examples of loop invariants: <https://arxiv.org/pdf/1211.4470>
- Blog post on proving correctness of algorithms: <https://www.dinalherath.com/2017/Proving-Correctness-of-Algorithms/>

In this question, we shall explore loop invariants. Consider the following algorithm:

function FUSE(n, m)

Require: n and m are positive integers.

- 1: Initialise a new empty array A
 - 2: $a \leftarrow 2n$
 - 3: $b \leftarrow m$
 - 4: **while** $a > 1$ **do**
 - 5: $a \leftarrow \lfloor \frac{a}{2} \rfloor$
 - 6: **if** a is odd **then**
 - 7: Append array A with new entry with value b
 - 8: $b \leftarrow 2b$
 - 9: **return** sum of entries of A
-

- (i) Given positive integers n and m , let val be the value returned from running FUSE(n, m). Express val in terms of n and m . (To get full credit, your expression must be as simple as possible.) [1 mark]
- (ii) Mimicking the style of the loop invariant proof for MAXVALUE, prove that your proposed expression from part (i) is indeed the value returned from the execution of FUSE(n, m). Your proof must explicitly state a loop invariant. [4 marks]

Hint 1: Your loop invariant should be a predicate $P = P(a, b, A)$ that depends on the loop variables a, b and A of the while-loop.

Hint 2: n is a positive integer. What is the binary representation of n ?

Hint 3: The predicate $P(a, b, A)$ would be a statement that is true, when variables a, b, A are substituted with the values they have at either the start or the end of each iteration of the while-loop. Be very careful with indices!

Hint 4: Can you find a predicate that involves $\lfloor \log_2(2n) \rfloor - \lfloor \log_2(a) \rfloor$? It is possible to express $P(a, b, A)$ as a “compound” predicate with three parts:

- the first part expresses the sum of entries of A in terms of $\lfloor \log_2(2n) \rfloor - \lfloor \log_2(a) \rfloor$;
- the second part expresses b in terms of $\lfloor \log_2(2n) \rfloor - \lfloor \log_2(a) \rfloor$;
- the third part expresses a in terms of $\lfloor \log_2(2n) \rfloor - \lfloor \log_2(a) \rfloor$.

Solution. (i) The output value val equals the product nm . [1 mark for correct answer]
(ii) Given input positive integers n and m , let $d := \lfloor \log_2(n) \rfloor$. (Note that once input integer n is fixed, the value of d would be fixed.) We can write integer n as

$$n = \sum_{k=0}^d \alpha_k 2^k = \alpha_d \cdot 2^d + \alpha_{d-1} \cdot 2^{d-1} + \cdots + \alpha_1 \cdot 2 + \alpha_0$$

for some uniquely determined integers $\alpha_0, \alpha_1, \dots, \alpha_d$ in $\{0, 1\}$, where $\alpha_d = 1$.

[0.5 mark for setting up notation for digits of binary representation of integer n]
Typically, the string $\alpha_d \alpha_{d-1} \cdots \alpha_1 \alpha_0$ is called the binary representation of the integer n , and $\alpha_0, \alpha_1, \dots, \alpha_d$ are called the digits of this binary representation of n . (We will learn more about number representations in Week 10).

Define $\alpha_{-1} := 0$, and for each $0 \leq j \leq d+1$, let us define the following integers

$$r_j := \sum_{k=j-1}^d \alpha_k 2^{k+1-j};$$

$$s_j := \sum_{k=0}^{j-1} \alpha_k 2^k.$$

In particular, note that $s_0 = 0$, $r_0 = 2n$, and $n = 2^j r_{j+1} + s_j$ for all $0 \leq j \leq d$.

[0.5 mark for considering $\sum_{k=j}^d \alpha_k 2^{k+1-j}$ or $\sum_{k=0}^{j-1} \alpha_k 2^j$ or something analogous.]

Let $P = P(a, b, A)$ be the predicate given by

$$\begin{aligned} &(\text{sum of entries of array } A) = s_{(d+1-\lfloor \log_2(a) \rfloor)} m, \\ &\text{and } b = 2^{(d+1-\lfloor \log_2(a) \rfloor)} m, \text{ and } a = r_{(d+1-\lfloor \log_2(a) \rfloor)}. \end{aligned}$$

where the domain of discourse for P is the set of all triples (a, b, A) such that $a \in \{1, \dots, n\}$, b is an integer at least m , and A is an integer array.

[1 mark for correct predicate $P(a, b, A)$, 0.5 mark for partially correct predicate $P(a, b, A)$]

- **Initialization:** Before the first iteration of the while-loop, the value of a is initialized as $a = 2n$, the value of b is initialized as $b = m$, and the value of array A is initialized as the empty array, whose sum of entries equals 0. Since $\lfloor \log_2(2n) \rfloor = d+1$, and since $s_0 = 0$, proposition $P(2n, m, [])$ says the following:

The sum of entries of $[]$ equals $s_0 m = 0$, and $b = 2^0 m = m$, and $a = r_0 = 2n$.

Thus, the proposition $P(2n, m, [])$ is true, which means that $P(a, b, A)$ is true prior to the first iteration. [0.5 mark for correct initialization.]

- **Maintenance:** Assume that $P(a, b, A)$ is true immediately before the execution of an iteration of the while-loop, based on the values of a, b, A at the start of the iteration. Our goal is to show that $P(a, b, A)$ is true immediately after the execution of this iteration, based on the values of a, b, A at the end of the iteration.

Let $a = p$ and $b = q$ be the values of a and b , respectively, at the start of the iteration. Let val be the value of the sum of entries of the array A at the start of the iteration. Since $P(a, b, A)$ is true at the start of the iteration, it means that p, q, val satisfy the following

- $val = s_{(d+1-\lfloor \log_2(p) \rfloor)}m$.
- $q = 2^{(d+1-\lfloor \log_2(p) \rfloor)}m$.
- $p = r_{(d+1-\lfloor \log_2(p) \rfloor)}$.

After running line 5, the value of a changes from

$$a = p = r_{(d+1-\lfloor \log_2(p) \rfloor)} = \sum_{k=d-\lfloor \log_2(p) \rfloor}^d \alpha_k 2^{(k-(d-\lfloor \log_2(p) \rfloor))}$$

to

$$a = \lfloor \frac{p}{2} \rfloor = \sum_{k=d+1-\lfloor \log_2(p) \rfloor}^d \alpha_k 2^{(k-(d-\lfloor \log_2(p) \rfloor)+1)} = r_{(d+2-\lfloor \log_2(p) \rfloor)}.$$

This means the updated value of a is odd if and only if $\alpha_{(d+1-\lfloor \log_2(p) \rfloor)} = 1$.

After running lines 6 and 7, the sum of entries of the array A would become

$$\begin{aligned} val + \alpha_{(d+1-\lfloor \log_2(p) \rfloor)}q &= \left(s_{(d+1-\lfloor \log_2(p) \rfloor)} + \alpha_{(d+1-\lfloor \log_2(p) \rfloor)} 2^{(d+1-\lfloor \log_2(p) \rfloor)} \right) m \\ &= \left(s_{(d+2-\lfloor \log_2(p) \rfloor)} \right) m \end{aligned}$$

Finally, after running line 8, the value of b is updated from $b = q = 2^{(d+1-\lfloor \log_2(p) \rfloor)}m$ to $b = 2q = 2^{(d+2-\lfloor \log_2(p) \rfloor)}m$. Thus, we conclude that $P(a, b, A)$ is true immediately after the execution of this iteration. [1 mark for correct maintenance.]

- **Termination:** Since a is always an integer throughout the while-loop, we know that if $a > 1$ at the start of an iteration, then $a \geq 2$ at the start of that iteration, and a gets updated to some value ≥ 1 at the end of that iteration. Hence, we exit the while-loop after the value of a reaches $a = 1$ at the end of some iteration.

Let $b = b_{\text{final}}$ be the value of b at the end of the last iteration. Also, let $A = A_{\text{final}}$ be the value of array A at the end of the last iteration, and let val_{final} be the sum of the entries of array A_{final} . Since proposition $P(1, b_{\text{final}}, A_{\text{final}})$ is true after exiting the while-loop, and since $(d+1-\lfloor \log_2(1) \rfloor) = d+1$, we get that

$$val_{\text{final}} = s_{d+1}m = nm.$$

Finally, note that line 9 of the pseudocode tells us that the value $val_{\text{final}} = nm$ is returned as the output to $\text{FUSE}(n, m)$. [0.5 mark for correct termination.]